

OPTIMIZANDO LA RED NEURONAL

Ignacio Gómez
2025

Contenido

- Reducir el overfitting
 - Data Augmentation
 - Regularizacion
 - Batch Normalization
 - Early Stopping
- Inicialización de los pesos
- Batches
- Optimizar los hiperparámetros
- Data Augmentation para nivelar clases no balanceadas (SMOTE)

La optimización de la red es un proceso iterativo

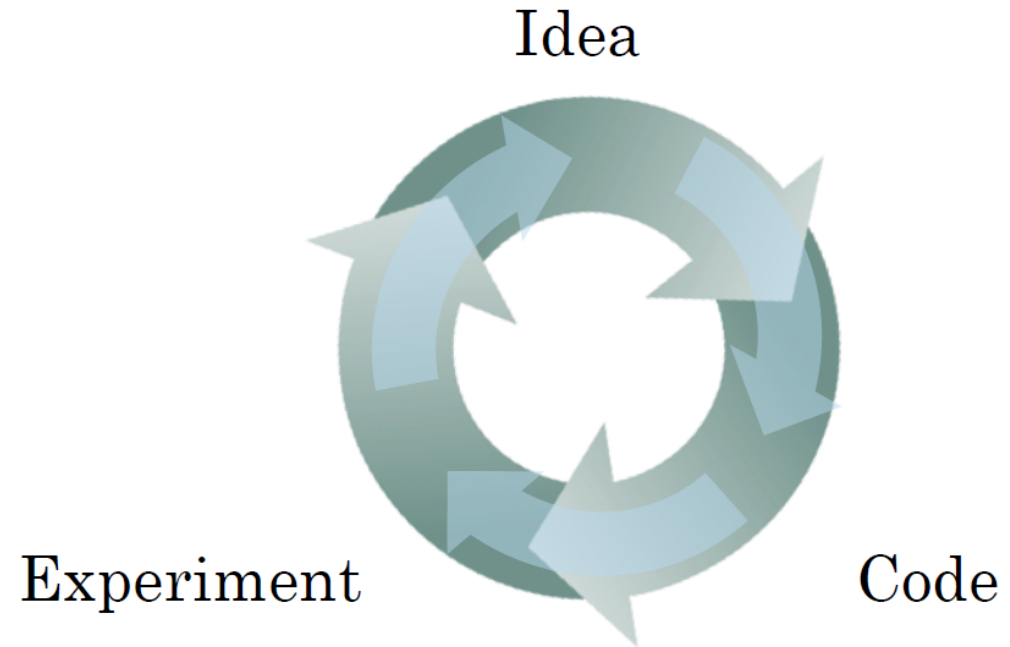
layers

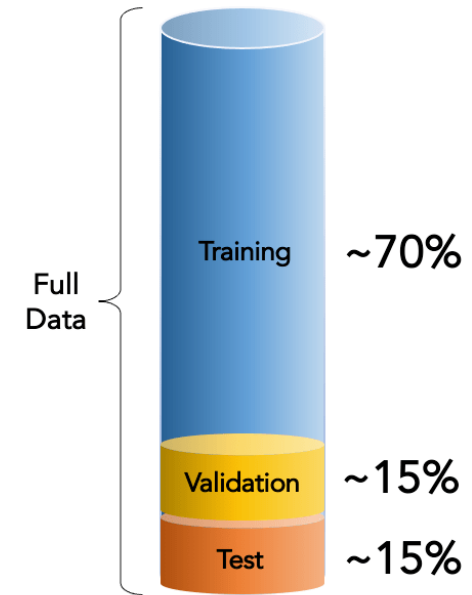
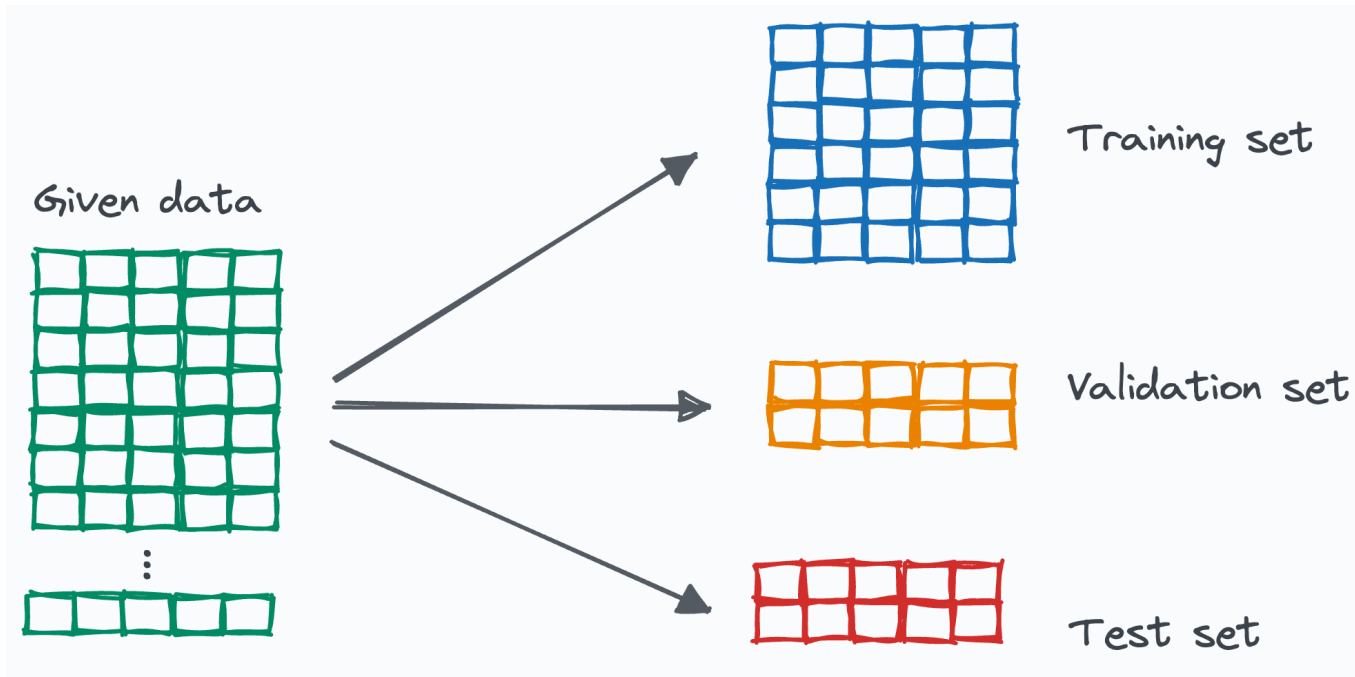
hidden units

learning rates

activation functions

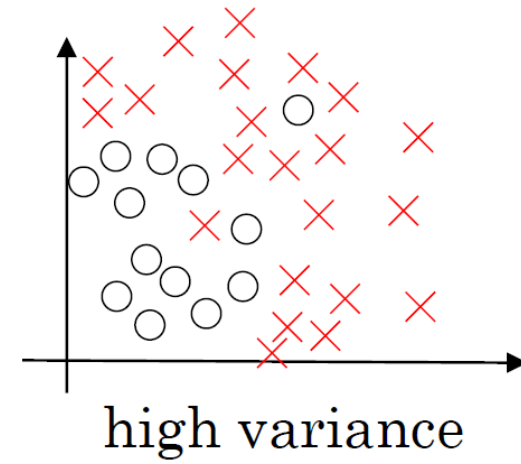
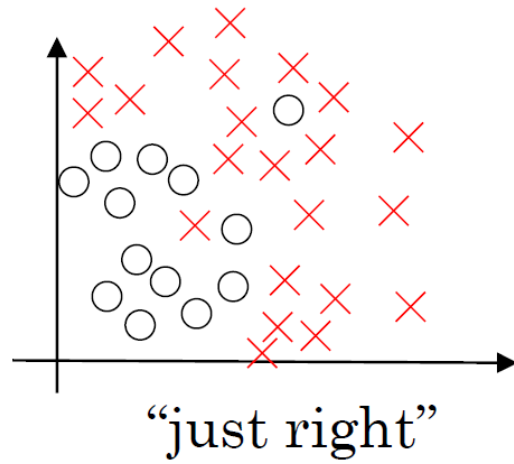
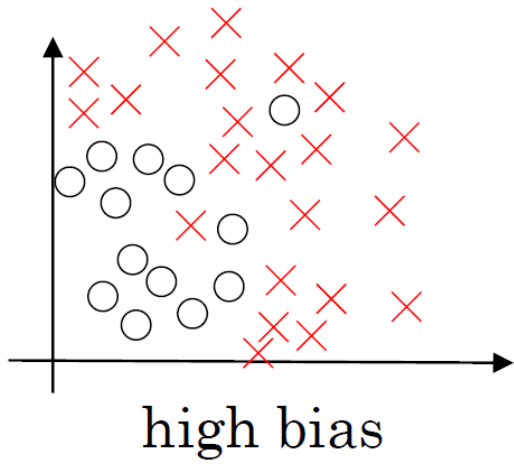
...





En Redes Neuronales se suele utilizar un número de casos de prueba muy grande, por lo que el tradicional split 70% training 30% test no es adecuado. Valores típicos pueden ser:

$N=1.000.000$ casos $\rightarrow$ $N_{train}=980.000$ $N_{dev}=10.000$ $N_{test}=10.000$

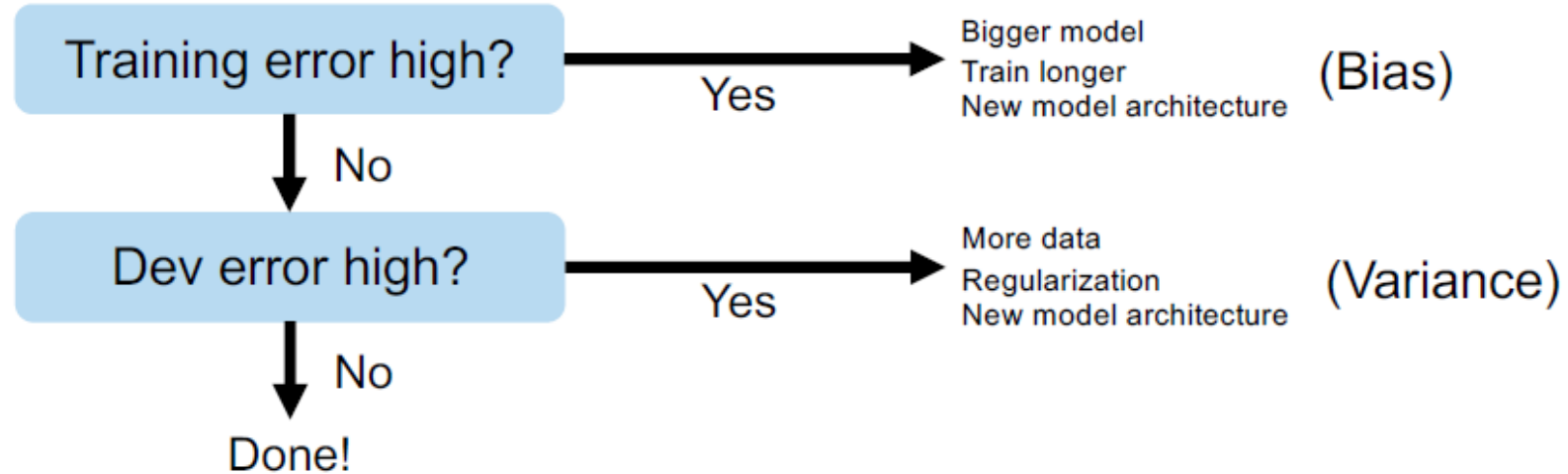


Train Acc	
Dev Acc	

Train Acc	
Dev Acc	

Train Acc	
Dev Acc	

Basic recipe for machine learning



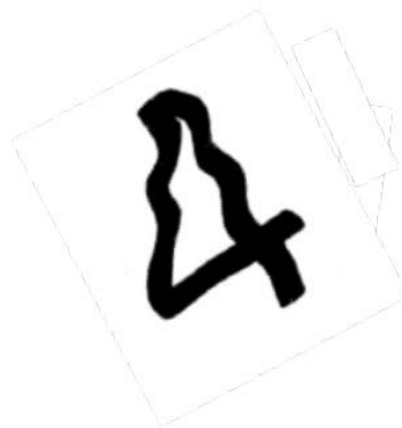
Reduciendo el Overfitting

- Utilizar más datos de entrenamiento. Tener más datos siempre mejora, no tiene efectos secundarios salvo el tiempo y el dinero.
- Si no tenemos más datos, o es demasiado costoso conseguirlos, utilizar *Data Augmentation*.
- Utilizar una arquitectura de red más simple. Con frecuencia difícil de ajustar, porque si te pasas generas underfitting.
- Regularización. En redes neuronales se hace de dos maneras: Regularización L2 o *dropout*.
- Early Stopping.

Data Augmentation



4



Audio Data Augmentation

Noise injection: add gaussian or random noise to the audio dataset to improve the model performance.

Shifting: shift audio left (fast forward) or right with random seconds.

Changing the speed: stretches times series by a fixed rate.

Changing the pitch: randomly change the pitch of the audio.

Text Data Augmentation

Word or sentence shuffling: randomly changing the position of a word or sentence.

Word replacement: replace words with synonyms.

Syntax-tree manipulation: paraphrase the sentence using the same word.

Random word insertion: inserts words at random.

Random word deletion: deletes words at random.

Image Augmentation

Geometric transformations: randomly flip, crop, rotate, stretch, and zoom images. You need to be careful about applying multiple transformations on the same images, as this can reduce model performance.

Color space transformations: randomly change RGB color channels, contrast, and brightness.

Kernel filters: randomly change the sharpness or blurring of the image.

Random erasing: delete some part of the initial image.

Mixing images: blending and mixing multiple images.

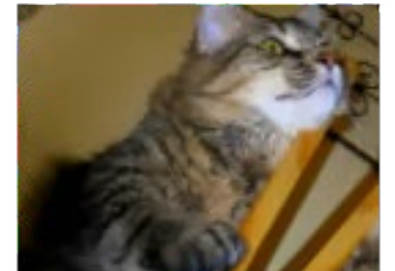
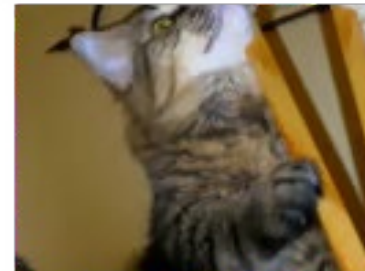
Advanced Techniques

Generative adversarial networks (GANs): used to generate new data points or images. It does not require existing data to generate synthetic data.

Neural Style Transfer: a series of convolutional layers trained to deconstruct images and separate context and style.

Data Augmentation en Keras

```
data_augmentation = keras.Sequential([
    layers.RandomFlip("horizontal_and_vertical"),
    layers.RandomRotation(0.4),
])
plt.figure(figsize=(8, 7))
for i in range(6):
    augmented_image = data_augmentation(image)
    ax = plt.subplot(2, 3, i + 1)
    plt.imshow(augmented_image.numpy()/255)
    plt.axis("off")
```



Data Augmentation en Keras

```
model = keras.Sequential([  
    # Add the preprocessing layers you created earlier.  
    resize_and_rescale,  
    data_augmentation,  
    # Add the model layers  
    layers.Conv2D(16, 3, padding='same', activation='relu'),  
    layers.MaxPooling2D(),  
    layers.Flatten(),  
    layers.Dense(128, activation='relu'),  
    layers.Dense(64, activation='relu'),  
    layers.Dense(1, activation='sigmoid')  
])
```

También se puede añadir una capa de data augmentation directamente en la arquitectura de la red

Regularización

Igual que en otros modelos de ML, consiste en añadir a la función de pérdida un término adicional que penaliza los valores grandes de los pesos. Por ejemplo, para un problema de clasificación binaria y regularización L2:

$$L_{\text{total}} = -\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)] + \frac{\lambda}{2} \sum_j w_j^2$$

Al realizar el proceso de backpropagation, esto se convierte en un decaimiento progresivo de los pesos (weight decay):

$$w_j \leftarrow w_j - \eta \left(\frac{\partial L(\text{datos})}{\partial w_j} + \lambda w_j \right)$$

Donde:

- η : Tasa de aprendizaje.
- λ : Hiperparámetro de regularización.

Regularización L2 con Keras

- **Regularización L1-L2:** Keras permite usar L1 o L2, o una combinación de ambos (L1+L2) mediante el argumento `kernel_regularizer` al definir capas.

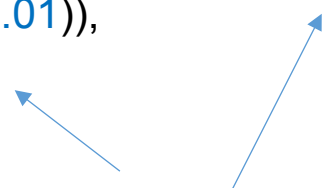
```
from tensorflow.keras import regularizers
```

```
model = Sequential([
```

```
Dense(64, input_dim=100, activation='relu', kernel_regularizer=regularizers.l2(0.01)), # L2 regularizacion
```

```
Dense(32, activation='relu', kernel_regularizer=regularizers.l1(0.01)), # L1 regularizacion
```

```
Dense(1, activation='sigmoid') ])
```



Aquí, $\lambda=0.01$ es el coeficiente de regularización.

- Se puede especificar el regularizador para cada capa. Puedes aplicar diferentes tipos o intensidades de regularización a distintas capas.
- Los sesgos se regularizan por separado con `bias_regularizer` si se desea. No se suele hacer.
- Cuaderno: `Regularizacion.ipynb` (Colab)

Regularización con *dropout* con Keras

- **Dropout:** Apaga aleatoriamente un porcentaje de neuronas en cada paso del entrenamiento, forzando al modelo a aprender representaciones más generalizadas.

```
from tensorflow.keras.layers import Dropout
```

```
model = Sequential([
```

```
Dense(64, input_dim=100, activation='relu'),
```

```
Dropout(0.5), # Apaga el 50% de las neuronas durante el entrenamiento
```

```
Dense(32, activation='relu'),
```

```
Dropout(0.3), # Apaga el 30% de las neuronas
```

```
Dense(1, activation='sigmoid') ])
```

Elección de rate:

- $p=0.5$ es común en capas ocultas.
- p más bajo (e.g., 0.2) puede ser usado cerca de las entradas.

- En cada paso de entrenamiento (forward pass), se desactiva aleatoriamente una proporción predefinida de neuronas en cada capa (independientemente para cada iteración).
- Actúa como un método de combinación de modelos (ensemble), ya que la red aprende múltiples subredes diferentes durante el entrenamiento.
- Cuaderno: `Regularization.ipynb` (Colab)

- **Batch Normalization:** Normaliza las activaciones (z_i^l) de cada capa durante el entrenamiento, reduciendo el desplazamiento interno de covarianza (internal covariate shift) y mejorando la estabilidad y velocidad del entrenamiento.

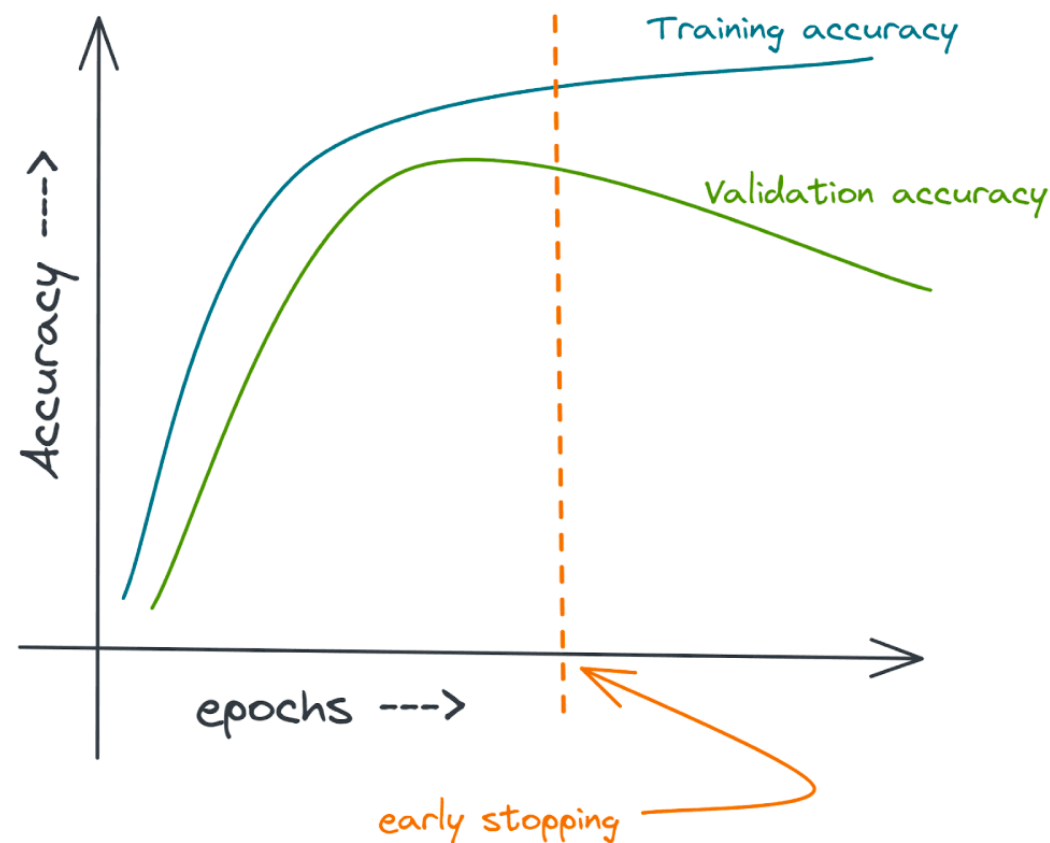
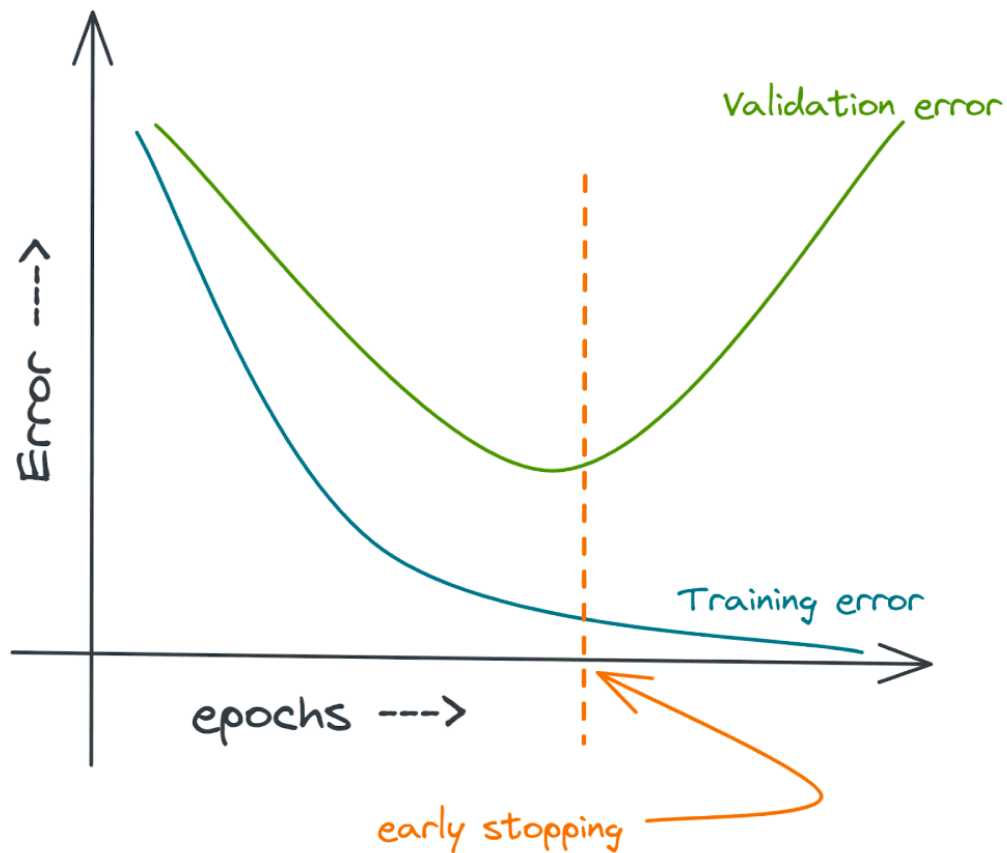
```
from tensorflow.keras.layers import Dense, BatchNormalization,
Activation
```

```
model = Sequential([ Dense(128, input_shape=(784,)),
    BatchNormalization(),           # Normalización de batch
    Activation('relu'),           # Función de activación
    Dense(64),
    BatchNormalization(),
    Activation('relu'),
    Dense(10, activation='softmax') # Capa de salida
])
```

- Aumenta la velocidad de entrenamiento.
- Permite usar tasas de aprendizaje más altas.
- Actúa como una forma de regularización, reduciendo el sobreajuste.

- Durante el entrenamiento, los pesos de la red se actualizan iterativamente, lo que provoca cambios en las distribuciones de las activaciones intermedias. Esto puede ralentizar el aprendizaje y hacer más difícil ajustar hiperparámetros.
- BN mitiga este problema al mantener las activaciones normalizadas en cada mini-batch, estabilizando el aprendizaje
- Cuaderno: Regularization.ipynb (Colab)

Early Stopping



Consiste en parar el entrenamiento en el momento en el que el error de validación empieza a aumentar (o la precisión a aumentar en problemas de clasificación). El problema es que en la práctica las curvas nunca son tan limpias.

Early Stopping en Keras

```
from keras.callbacks import EarlyStopping

# Define early stopping callback
early_stopping = EarlyStopping(
    monitor='val_loss', # Monitor validation loss
    patience=5,          # Stop after 5 epochs of no improvement
    restore_best_weights=True # Restore the best weights
)

# Train the model
history = model.fit(
    x_train, y_train,
    epochs=50, # Maximum number of epochs
    batch_size=64,
    validation_split=0.2, # Use 20% of the training data as validation data
    callbacks=[early_stopping] # Add early stopping callback
)
```

El parámetro paciencia hace exactamente eso: tener paciencia. Es para evitar que el entrenamiento se pare en la primera oscilación.

Cuaderno: EarlyStopping.ipynb

Guardar mejor modelo

```
from keras.callbacks import ModelCheckpoint

# Define el callback para guardar los mejores pesos
checkpoint_callback = ModelCheckpoint(
    filepath='mejor_modelo.h5', # Ruta donde se guardará el modelo
    monitor='val_loss',         # Métrica a monitorizar (pérdida en validación)
    save_best_only=True,        # Guardar solo el mejor modelo
    save_weights_only=True,     # Guardar solo los pesos, no todo el modelo
    mode='min',                 # Minimizar la métrica (val_loss)
    verbose=1                   # Mostrar mensajes
)

# Entrena el modelo con el callback
model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=100,
    callbacks=[checkpoint_callback]
)
```

Inicialización de los pesos

En regresión logística los pesos se inicializan a cero antes del entrenamiento, pero en ANN, esto produce que la red no aprenda.

Formas de inicializar los pesos:

1) Al azar, según una distribución de probabilidad:

```
initializer = RandomNormal(mean=0.0, stddev=1.0)  
layer = Dense(3, kernel_initializer=initializer)
```

```
initializer = RandomUniform(minval=0.0, maxval=1.0)  
layer = Dense(3, kernel_initializer=initializer)
```

2) Método Glorot o Xavier: extraer los pesos de una distribución normal de media cero y una varianza que depende del número de neuronas de cada capa.

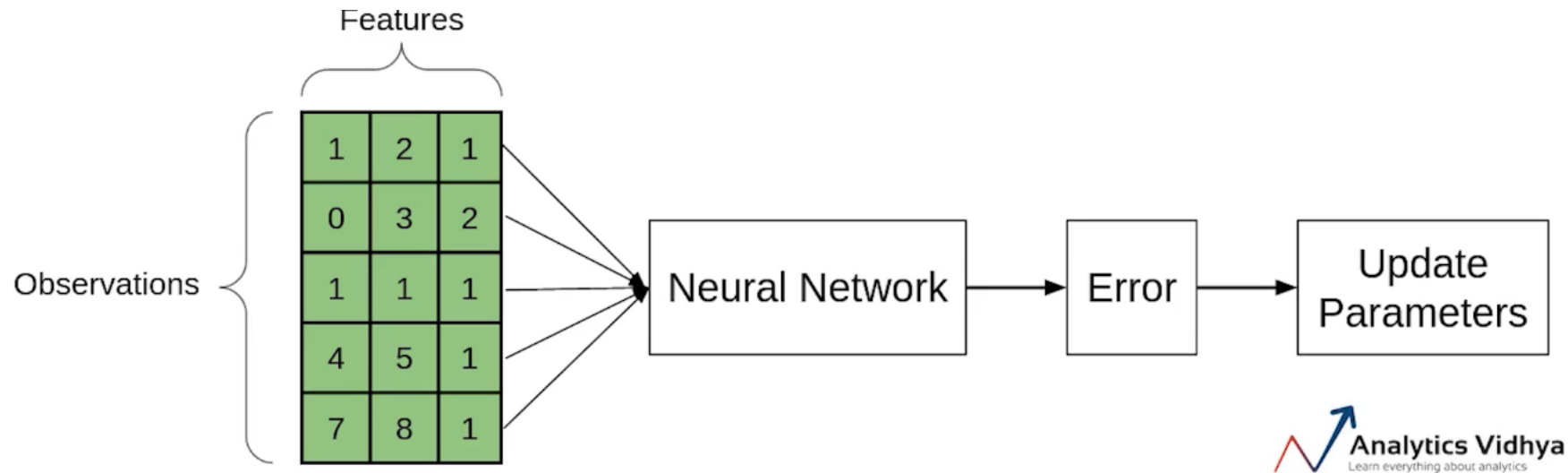
```
initializer = GlorotNormal()  
layer = Dense(3, kernel_initializer=initializer)
```

3) He Normal. Muy parecida, con una varianza distinta, pero del mismo estilo.

4) Cuaderno: `InitializationKeras.ipynb`

Batches

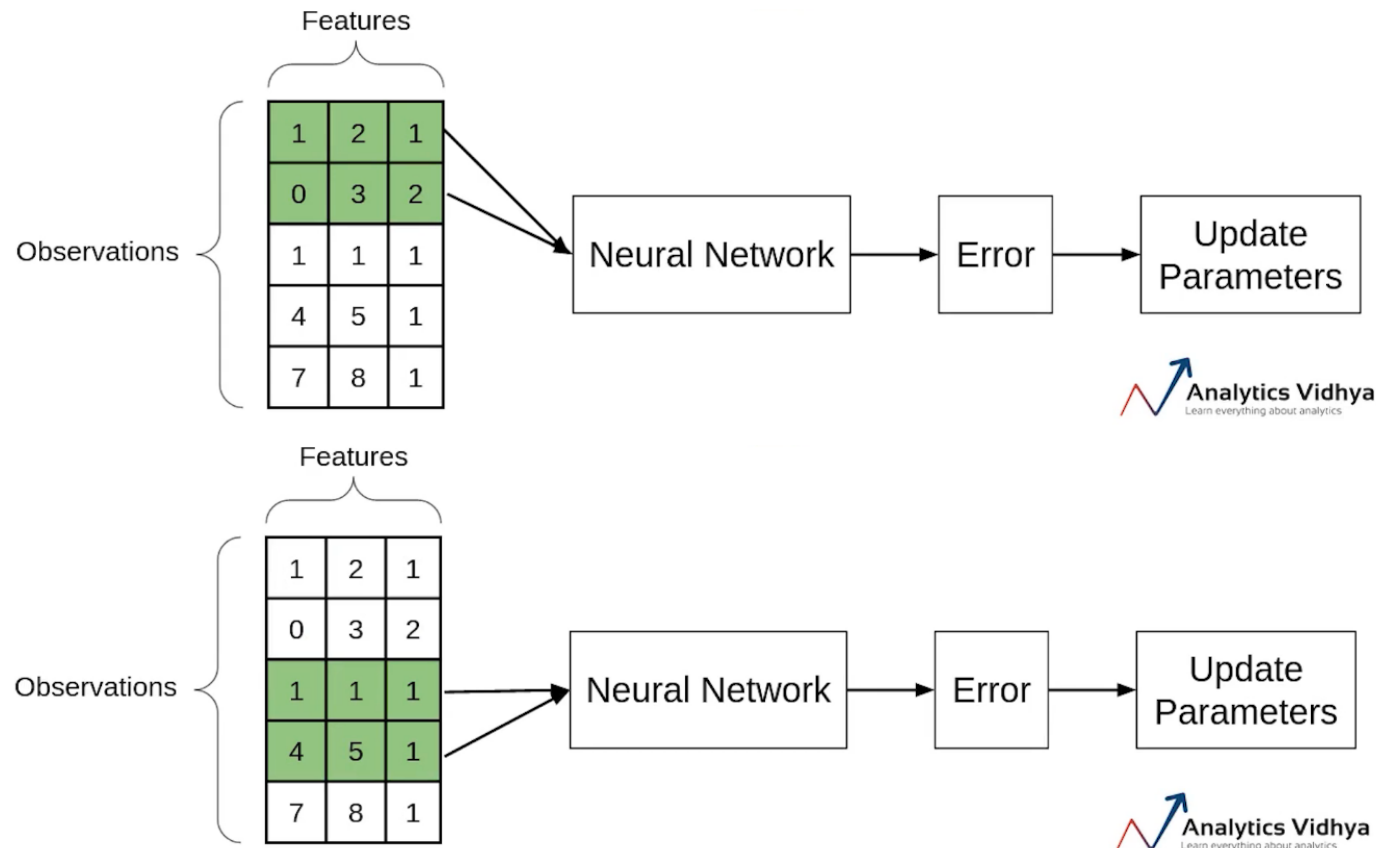
Hemos dicho que, en una ANN, todos los casos de entrenamiento se procesan a la vez, en una pasada hacia adelante y otra hacia atrás, para después dar un pequeño paso en la dirección contraria al gradiente:



Pero si tenemos 1 millón de observaciones, hay que manejar una matriz inmensa (probablemente ni siquiera cabrá en memoria) para obtener un simple paso en el algoritmo de optimización. Hay otra manera más eficiente de hacerlo.

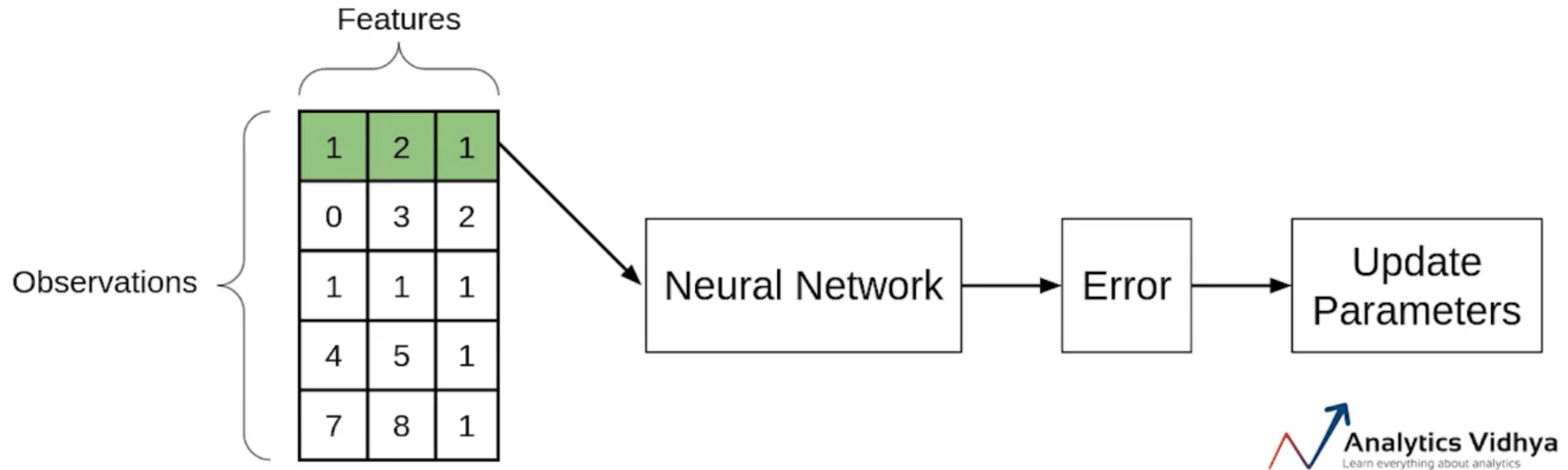
Mini Batches

En el descenso por mini batches (o mini lotes), procesamos solo un subconjunto del total de casos de entrenamiento, y actualizamos los valores de los parámetros, a continuación, otro subconjunto, y así hasta que hemos completado el dataset completo, en ese momento hemos completado una iteración o época.



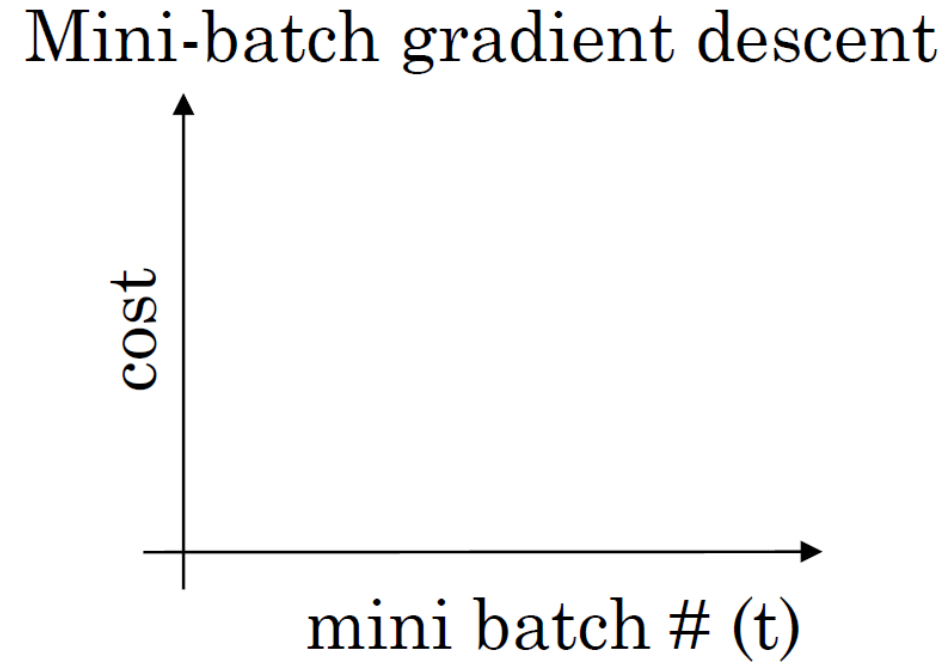
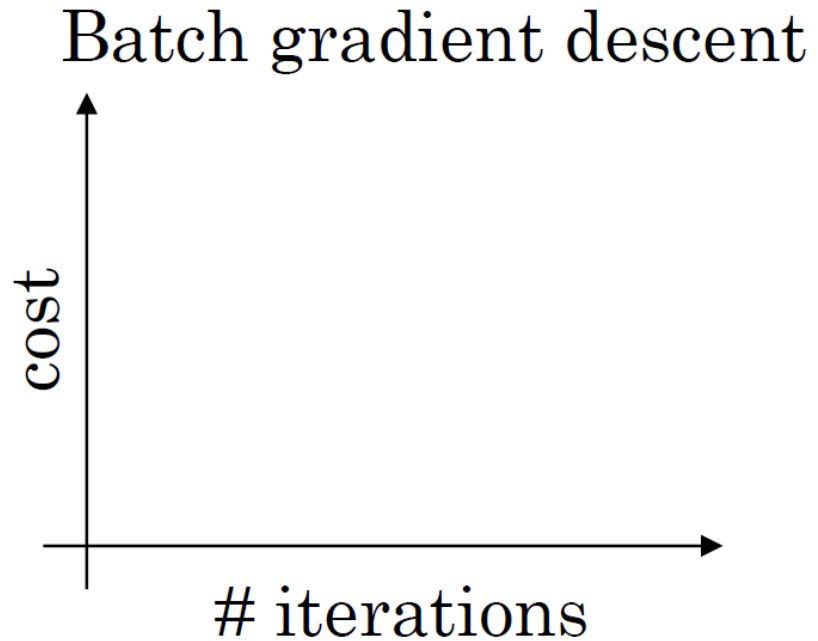
Descenso de gradiente estocástico

Es el caso extremo en que hacemos el tamaño de lote igual a uno.



Mini Batches

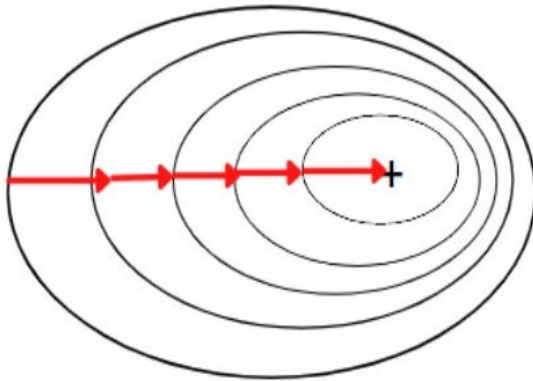
¿Cuál es el efecto de los mini batches en el algoritmo de descenso del gradiente:



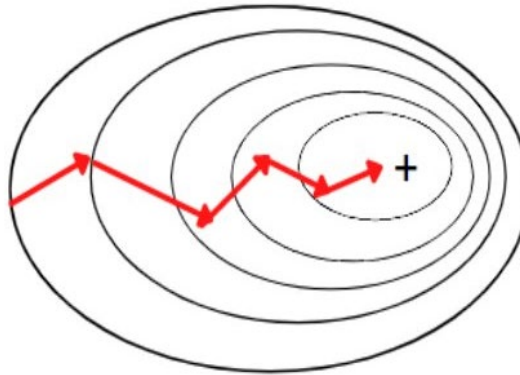
Mini Batches

¿Cuál es el efecto de los mini batches en el algoritmo de descenso del gradiente:

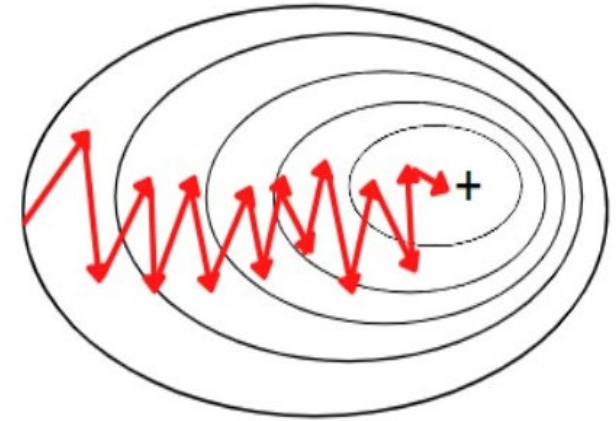
Batch Gradient Descent



Mini-Batch Gradient Descent



Stochastic Gradient Descent



Mini Batches. Tamaños

¿Cómo seleccionamos el tamaño adecuado del mini batch?

- 1) $N=m$ Descenso por lotes. Solo lo usamos para dataset pequeños, algunos miles.
- 2) $N=1$ Descenso de gradiente estocástico. Demasiado lento. A veces se utiliza en aprendizaje en línea.
- 3) $N=p$ Mini batches. Valores típicos son 64, 128, 256, 512.

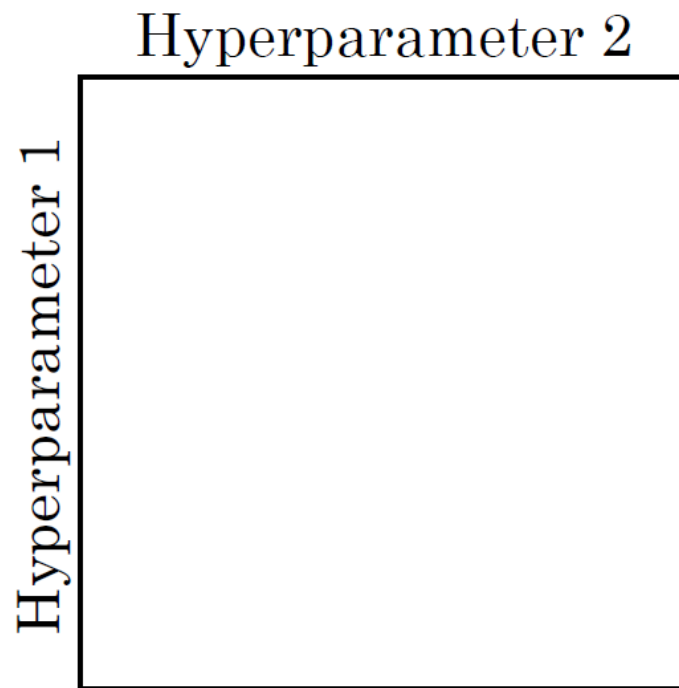
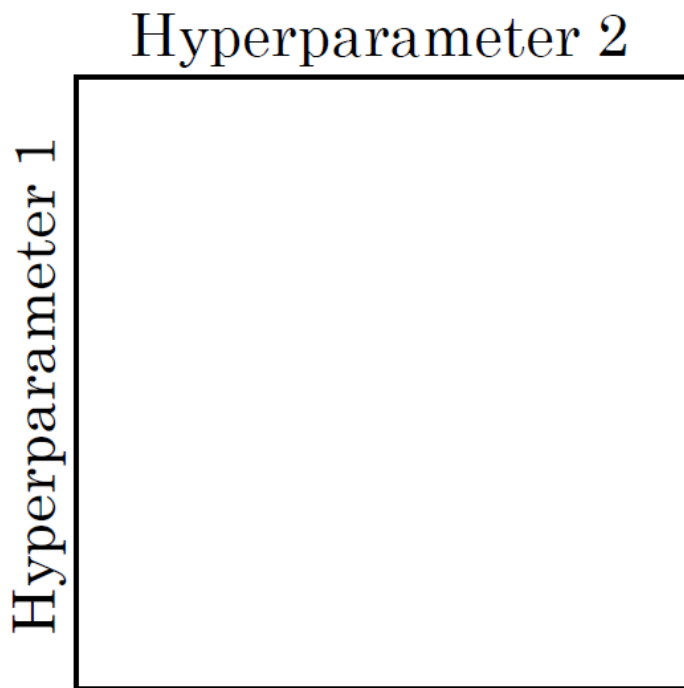
Cuaderno: `Batches.ipynb`

Optimizar los hiperparámetros

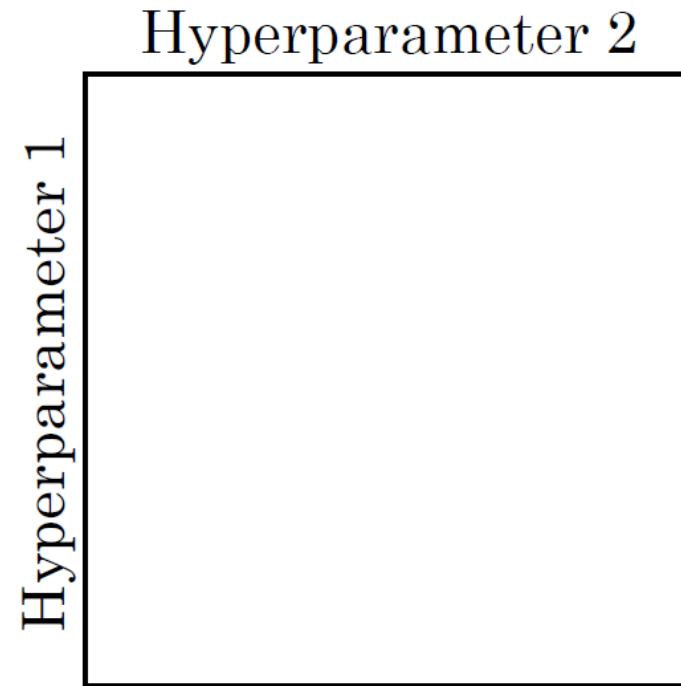
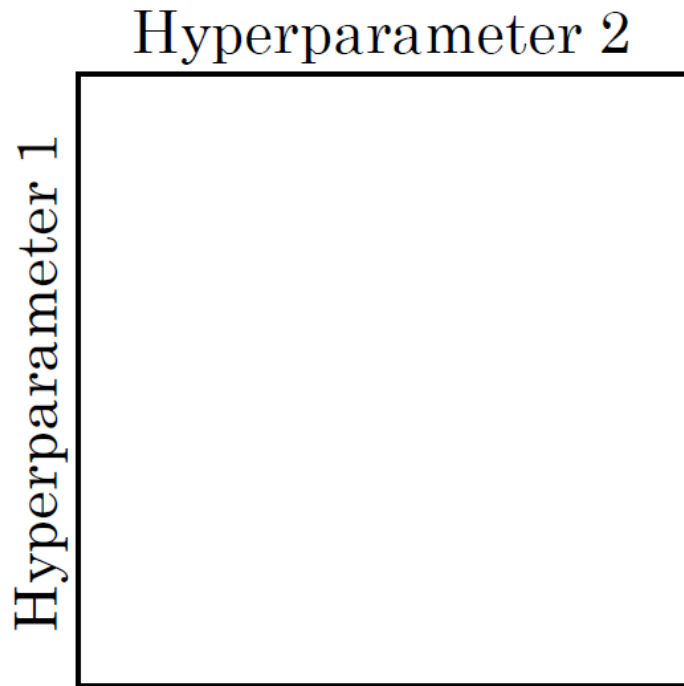
Al final, en una ANN tenemos un montón de hiperparámetros y pocas pistas para seleccionar sus valores, salvo prueba y error:

- Número de capas
- Número de neuronas por capa
- Learning Rate
- Parámetros λ de la regularización de cada capa
- Número de épocas
- Tamaño de los batches
- Etc..

No utilizar una cuadrícula sino muestrear



Ir refinando los valores de muestreo



Además, hay que tener en cuenta que algunos parámetros, como la tasa de aprendizaje deberíamos variarlos exponencialmente: 0.1 0.01 0.001 ...

Dos aproximaciones en la optimización de modelos

Babysitting one
model



Panda

Training many
models in parallel



Caviar

Cuaderno: `Tunning.ipynb`

SMOTE

Los clasificadores no funcionan bien con clases desbalanceadas. El algoritmo aprende que su rendimiento global no se ve muy influenciado por las clases con pocos casos, y tiende a ignorarlas.

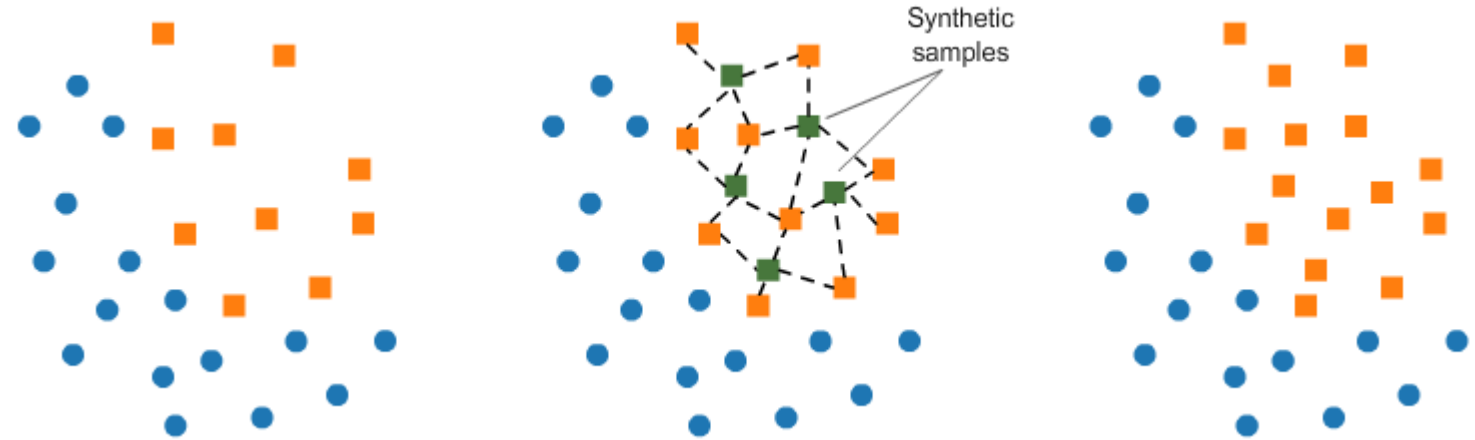
Una manera de solucionar este problema es añadir casos sintéticos a las clases con menos casos de entrenamiento hasta que todas las clases tengan aproximadamente el mismo número de casos.

Una técnica habitual para hacer esto se denomina SMOTE: (Synthetic Minority Over-sampling Technique).

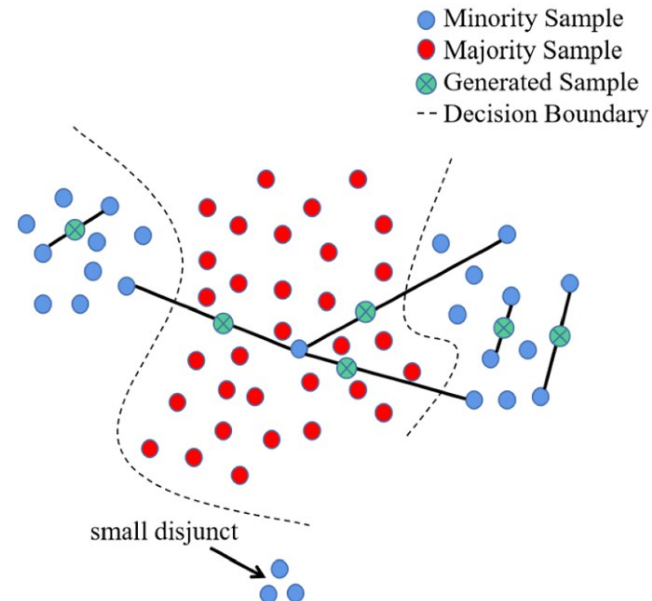
SMOTE genera muestras sintéticas de la clase minoritaria para equilibrar el conjunto de datos. En lugar de simplemente duplicar las muestras existentes de la clase minoritaria (lo que podría causar sobreajuste), SMOTE crea nuevas instancias sintéticas combinando características de muestras existentes de la clase minoritaria. Esto ayuda a mejorar la generalización del modelo.

SMOTE

Consiste en generar nuevos casos con combinaciones de los casos existentes:



Es obvio que no va a funcionar siempre:



Cuaderno: SMOTE.ipynb

Salvando nuestro modelo

Podemos guardar nuestros modelos entrenados para hacer inferencia o incluso para continuar el entrenamiento. Podemos hacerlo con herramientas de Keras o herramientas más generales como ONNX.

Con Keras:

```
model.save('mi_modelo.h5')
```

Y para utilizarlo posteriormente:

```
from keras.models import load_model
```

```
model = load_model('mi_modelo.h5')
```

```
predictions = model.predict(nuevos_datos)
```

Salvando nuestro modelo

También podemos guardar solo los pesos y cargarlos en un modelo ya creado:

```
model.save_weights('mis_pesos.h5')
```

Y para utilizarlo posteriormente:

```
model = Sequential([  
    Dense(64, activation='relu', input_shape=(input_dim,)),  
    Dense(10, activation='softmax')  
])
```

```
model.load_weights('mis_pesos.h5')
```

```
model.compile(optimizer='adam', loss='categorical_crossentropy',  
metrics=['accuracy'])
```

```
predictions = model.predict(nuevos_datos)
```

ONNX

ONNX (Open Neural Network Exchange) es un formato abierto y estándar para representar modelos de aprendizaje automático (machine learning). Fue creado para facilitar la interoperabilidad entre diferentes frameworks de deep learning y herramientas de inferencia.

Ventajas:

Portabilidad:

Puedes entrenar un modelo en un framework (por ejemplo, PyTorch) y ejecutarlo en otro (por ejemplo, TensorFlow) sin preocuparte por las diferencias entre los frameworks.

Despliegue en múltiples plataformas:

ONNX permite ejecutar modelos en una variedad de hardware y entornos, desde servidores en la nube hasta dispositivos edge (como móviles o IoT).

Optimización para inferencia:

ONNX Runtime y otras herramientas compatibles con ONNX ofrecen optimizaciones avanzadas para acelerar la inferencia en diferentes hardware.

Estándar abierto:

ONNX es un proyecto de código abierto respaldado por una comunidad activa y grandes empresas como Microsoft, Facebook (Meta), NVIDIA e Intel.

Salvando nuestro modelo ONNX

También podemos guardar solo los pesos y cargarlos en un modelo ya creado:

```
import tensorflow as tf
import tf2onnx

# Supongamos que ya tienes un modelo de Keras entrenado
model = tf.keras.models.load_model('mi_modelo.h5')

# Guardar el modelo en formato SavedModel (requerido por tf2onnx)
tf.saved_model.save(model, "mi_modelo_savedmodel")

# Convertir el modelo SavedModel a ONNX
onnx_model, _ = tf2onnx.convert.from_saved_model("mi_modelo_savedmodel",
output_path="mi_modelo.onnx")
```

ONNX Model Zoo

Ejecutando nuestro modelo ONNX

```
import onnxruntime as ort
import numpy as np

# Cargar el modelo ONNX
session = ort.InferenceSession("mi_modelo.onnx")

# Obtener los nombres de las entradas y salidas del modelo
input_name = session.get_inputs()[0].name
output_name = session.get_outputs()[0].name

# Preparar los datos de entrada (deben ser un numpy array con la forma correcta)
# Por ejemplo, si el modelo espera una entrada de forma (1, 28, 28, 1)
nuevos_datos = np.random.rand(1, 28, 28, 1).astype(np.float32) # Ejemplo de datos
aleatorios

# Realizar inferencia
predictions = session.run([output_name], {input_name: nuevos_datos})

# predictions contendrá las salidas del modelo
print(predictions)
```